

Rapport Projet Enron Discovery

Plateforme d'e-Discovery avec Django et PostgreSQL

TCHAGNAO Bahissou, WADE Abdou

19 mars 2026

Table des matières

1	Introduction	3
2	Modélisation Conceptuelle des Données (MCD)	3
2.1	Entités et Attributs	3
2.2	Relations Principales	4
3	Modèle Relationnel et Normalisation	4
3.1	Tables Principales	4
3.1.1	Table Employee	4
3.1.2	Table Folder	4
3.1.3	Table Email	4
3.1.4	Tables de Liaison M2M	4
3.1.5	Table Attachment	5
3.2	Justifications des Choix	5
4	Optimisations et Performances	5
4.1	Indexes	5
5	Pipeline d'Ingestion et Parsing	6
5.1	Architecture du Pipeline	6
5.2	Extrait de Parsing	6
5.3	Idempotence	6
6	Interface Web et Fonctionnalités	6
6.1	Recherche avancée	6
7	Templates et Interface Utilisateur	7
8	Configuration des Routes	7
9	Tests Unitaires	7
9.1	Configuration des tests	8
10	Infrastructure et Déploiement	8
10.1	Docker Compose	8
11	Sécurité et Configuration	8
11.1	Rate limiting	8
12	Limitations et Perspectives d'Amélioration	8
12.1	Limitations Actuelles	8

13 Conclusion

1 Introduction

Le projet Enron Discovery vise à concevoir une plateforme d'enquête numérique (e-Discovery) à partir du corpus textuel d'emails Enron. L'objectif est de transformer une masse de données non structurées (plus de 500 000 emails) pour permettre :

- La navigation dans les échanges (threads)
- La recherche d'informations critiques
- L'identification des acteurs clés
- La reconstruction des conversations
- L'analyse des réseaux d'influence

Ce rapport présente l'architecture complète du projet, depuis la modélisation relationnelle jusqu'à l'implémentation web, en passant par le pipeline d'ingestion, les optimisations, les tests unitaires et l'intégration continue.

Source des données : Enron Email Dataset (CMU)

<https://www.cs.cmu.edu/~enron/>

Stack technique :

- Backend et interface : Django 5.2
- Base de données : PostgreSQL 16 (Docker)
- Recherche plein texte : PostgreSQL FTS (SearchVector + index GIN)
- Parsing/ingestion : Python (`email`, `re`, `datetime`)
- Tests : Django TestCase
- CI/CD : GitHub Actions
- Versionnage : Git

2 Modélisation Conceptuelle des Données (MCD)

Pour bien comprendre l'architecture globale du projet, il est important de visualiser le cycle complet de traitement des emails depuis l'utilisateur jusqu'au serveur et le stockage en base de données, puis le retour vers l'interface web :

1. **Client / Navigateur** : L'utilisateur interagit avec l'interface web, effectue des recherches, parcourt les threads ou consulte le graphe d'influence.
2. **Serveur Django** : Les requêtes sont traitées par le framework Django, qui orchestre l'accès aux données, la logique métier et les vues HTML/CSS.
3. **Base de données PostgreSQL** : Les emails, employés, dossiers et pièces jointes sont stockés selon le modèle relationnel. Les index et les champs `search_vector` permettent des recherches rapides et efficaces.
4. **Pipeline d'ingestion** : Les fichiers `.eml` sont importés via des scripts de parsing qui nettoient, structurent et alimentent la base.
5. **Retour client** : Les résultats des recherches, visualisations et explorations de threads sont renvoyés au navigateur.

2.1 Entités et Attributs

Entité	Attributs principaux
Employee	email (unique), name
Folder	name
Email	message_id (unique), date, subject, body, search_vector
Attachment	filename, content_type, size

TABLE 1 – Entités principales et attributs

2.2 Relations Principales

- Un employé envoie des emails : relation `from_employee` (1,N)
- Un email peut être envoyé à plusieurs destinataires (`to_employees`, `cc_employees`, `bcc_employees`) : relation N,N
- Un email peut répondre à un autre email : relation `in_reply_to` (auto-référence, 0,1)
- Un email est classé dans un dossier : relation `folder` (N,1)
- Un email peut avoir plusieurs pièces jointes : relation `attachment` (1,N)

3 Modèle Relationnel et Normalisation

Le modèle relationnel est conçu selon les principes de normalisation (3NF) pour assurer l'intégrité des données et de bonnes performances.

3.1 Tables Principales

3.1.1 Table Employee

```
CREATE TABLE enron_employee (  
    id SERIAL PRIMARY KEY,  
    email VARCHAR(254) UNIQUE NOT NULL,  
    name VARCHAR(200)  
);
```

3.1.2 Table Folder

```
CREATE TABLE enron_folder (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(200) NOT NULL  
);
```

3.1.3 Table Email

```
CREATE TABLE enron_email (  
    id SERIAL PRIMARY KEY,  
    message_id VARCHAR(500) UNIQUE NOT NULL,  
    date TIMESTAMP,  
    subject TEXT,  
    body TEXT,  
    from_employee_id INTEGER NOT NULL REFERENCES enron_employee(id),  
    folder_id INTEGER REFERENCES enron_folder(id),  
    in_reply_to_id INTEGER REFERENCES enron_email(id) ON DELETE SET NULL,  
    search_vector TSVECTOR  
);
```

3.1.4 Tables de Liaison M2M

```
CREATE TABLE enron_email_to_employees (  
    id SERIAL PRIMARY KEY,  
    email_id INTEGER NOT NULL REFERENCES enron_email(id) ON DELETE CASCADE,  
    employee_id INTEGER NOT NULL REFERENCES enron_employee(id),  
    UNIQUE(email_id, employee_id)  
);  
  
CREATE TABLE enron_email_cc_employees (  
    id SERIAL PRIMARY KEY,  
    email_id INTEGER NOT NULL REFERENCES enron_email(id) ON DELETE CASCADE,  
    employee_id INTEGER NOT NULL REFERENCES enron_employee(id),  
    UNIQUE(email_id, employee_id)  
);
```

```

    id SERIAL PRIMARY KEY,
    email_id INTEGER NOT NULL REFERENCES enron_email(id) ON DELETE CASCADE,
    employee_id INTEGER NOT NULL REFERENCES enron_employee(id),
    UNIQUE(email_id, employee_id)
);

CREATE TABLE enron_email_bcc_employees (
    id SERIAL PRIMARY KEY,
    email_id INTEGER NOT NULL REFERENCES enron_email(id) ON DELETE CASCADE,
    employee_id INTEGER NOT NULL REFERENCES enron_employee(id),
    UNIQUE(email_id, employee_id)
);

```

3.1.5 Table Attachment

```

CREATE TABLE enron_attachment (
    id SERIAL PRIMARY KEY,
    email_id INTEGER NOT NULL REFERENCES enron_email(id) ON DELETE CASCADE,
    filename VARCHAR(500),
    content_type VARCHAR(200),
    size INTEGER
);

```

3.2 Justifications des Choix

- **Normalisation 3NF** : séparation des entités Employee et Email pour éviter la redondance ; un expéditeur ou un destinataire est représenté une seule fois.
- **Relations M2M** : tables dédiées To/Cc/Bcc.
- **Auto-référence** in_reply_to : reconstruction des fils.
- **Champ** search_vector : recherche plein texte PostgreSQL.
- **Références ON DELETE** : SET NULL sur in_reply_to, CASCADE sur tables dépendantes.

4 Optimisations et Performances

Type d'optimisation	Implémentation	Justification
Index B-Tree date	Sur date	Accélère tris et filtres
Index B-Tree message_id	Sur message_id	Vérification d'unicité
Index GIN	Sur search_vector	FTS rapide
select_related()	Clés étrangères	Réduit N+1
Pagination sans count()	_paginate_without_count()	Évite COUNT(*) coûteux
bulk_create	Lots de 5000	Import plus rapide
Idempotence	Via message_id	Pas de doublons

TABLE 2 – Stratégies d'optimisation

4.1 Indexes

```

models.Index(fields=["date"]),
models.Index(fields=["message_id"]),
GinIndex(fields=["search_vector"], name="email_fts_gin_idx"),

```

5 Pipeline d'Ingestion et Parsing

Le corpus Enron est fourni sous forme d'arborescence .eml. Le script `polls/management/commands/import_enr` assure l'import.

5.1 Architecture du Pipeline

1. Parcours récursif des fichiers.
2. Parsing des en-têtes (From/To/Cc/Bcc/Date/Subject/Message-ID/In-Reply-To).
3. Extraction du corps.
4. Nettoyage (signatures/disclaimers).
5. Validation des dates.
6. Création des entités.
7. Reconstruction des threads.
8. Mise à jour FTS.

5.2 Extrait de Parsing

```
def clean_body(self, body):
    if not body:
        return ""
    body = body.strip()
    body = re.split(r"\n[- ]*Original Message[- ]*\n",
                    body, flags=re.IGNORECASE)[0]
    body = re.split(r"\n--\s*\n", body)[0]
    disclaimer_patterns = [
        r"This e-mail.*?may contain confidential.*",
        r"Ce message.*?est confidentiel.*",
    ]
    for pattern in disclaimer_patterns:
        body = re.sub(pattern, "", body,
                      flags=re.IGNORECASE | re.DOTALL)
    return body.strip()
```

5.3 Idempotence

```
existing_ids = set(Email.objects.values_list("message_id", flat=True))
if message_id in existing_ids:
    already_exists += 1
    continue
```

6 Interface Web et Fonctionnalités

L'application expose 6 vues : accueil, dashboard, recherche, thread, influence, liste paginée.

6.1 Recherche avancée

```
def search_emails(request):
    q = request.GET.get("q", "").strip()
    sender = request.GET.get("sender", "").strip()
    date_from = request.GET.get("date_from", "")
    date_to = request.GET.get("date_to", "")
```

```

qs = Email.objects.select_related("from_employee").all()

if sender:
    qs = qs.filter(from_employee__email__icontains=sender)
if date_from:
    qs = qs.filter(date__date__gte=date_from)
if date_to:
    qs = qs.filter(date__date__lte=date_to)

if q:
    query = SearchQuery(q)
    qs = qs.filter(search_vector=query)\
        .annotate(rank=SearchRank("search_vector", query))\
        .order_by("-rank", "-date")
else:
    qs = qs.order_by("-date")

pagination = _paginate_without_count(qs, request.GET.get("page"))

context = {
    "emails": pagination["emails"],
    "current_page": pagination["current_page"],
    "has_previous": pagination["has_previous"],
    "has_next": pagination["has_next"],
    "q": q,
    "sender": sender,
    "date_from": date_from,
    "date_to": date_to,
    "total_results": _result_label(pagination),
}
return render(request, "search.html", context)

```

7 Templates et Interface Utilisateur

L'interface utilise Bootstrap 5. Tous les templates héritent de `base.html`.

8 Configuration des Routes

```

from django.urls import path
from . import views

app_name = 'enron'

urlpatterns = [
    path('', views.home, name='home'),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('search/', views.search_emails, name='search_emails'),
    path('thread/<int:email_id>/', views.thread_detail, name='thread_detail'),
    path('influence/', views.influence_graph, name='influence_graph'),
    path('emails/', views.email_list, name='email_list'),
]

```

9 Tests Unitaires

La suite de tests est dans `polls/tests.py`.

9.1 Configuration des tests

Les tests utilisent `django.test.TestCase` et une base de test Django dédiée. En CI, les tests tournent sur PostgreSQL 16.

```
python manage.py test polls -v 2
```

10 Infrastructure et Déploiement

10.1 Docker Compose

```
services:
  db:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_DB: enron_db
      POSTGRES_USER: enron
      POSTGRES_PASSWORD: enron2026
    ports:
      - "5433:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

11 Sécurité et Configuration

11.1 Rate limiting

```
from django_ratelimit.decorators import ratelimit

@ratelimit(key='ip', rate='100/h')
def search_emails(request):
    # Limité à 100 requêtes par IP et par heure
    ...
```

12 Limitations et Perspectives d'Amélioration

12.1 Limitations Actuelles

Limitation	Impact
Pas d'authentification	Accès public à tous les emails
Graphe d'influence statique	Tableau HTML non interactif
Pas d'export de données	Réutilisation limitée
Pas de rate-limiting	Vulnérable aux abus
Interface basique	UX minimale

TABLE 3 – Limitations identifiées

13 Conclusion

Ce projet met en œuvre une chaîne complète (Python, Django, PostgreSQL) pour transformer un corpus massif non structuré en plateforme e-Discovery exploitable.

- Modélisation relationnelle normalisée (3NF)
- Pipeline d'ingestion idempotent
- Recherche plein texte PostgreSQL (GIN + ranking)
- Interface web fonctionnelle
- Tests et CI automatisée

Résultat final : une plateforme e-Discovery fonctionnelle et documentée.